Teknologi Cerdas

Armein Z. R. Langi

2027-02-08

Table of contents

Preface

A highly effective project for a single semester is building a **Virtual AI (vAI) Customer Intelligence Engine**. This project moves students beyond traditional Business Intelligence (BI)—where dashboards simply report what happened—into the realm of Generative AI, where a virtual analyst explains *why* it happened and *what to do next*.

Here is how the project breaks down, integrating Kaggle datasets, traditional machine learning, and LLMs into a cohesive, semester-long build.

The Core Concept

Students will select a real-world Kaggle dataset focused on customer behavior (e.g., e-commerce purchasing habits, telecommunications churn, or bank credit card conversions). They will build a system that:

1. **Predicts:** Uses traditional ML to score which customers are most likely to buy or churn.
2. **Explains:** Uses an LLM (acting as the vAI) to read those predictions and generate personalized, plain-English strategies for the sales or marketing team.

The Tech Stack

Python provides the ideal foundation for this entire workflow, keeping the learning curve manageable for a single semester:

- **Data Wrangling:** Pandas and NumPy.
- **Predictive ML:** Scikit-learn (Random Forest or XGBoost).
- **The vAI / LLM:** Local machine learning frameworks like Ollama allow students to run models securely and cost-effectively on their own machines without managing cloud API keys.
- **Dashboard UI:** Streamlit allows students to deploy their Python scripts directly into interactive web applications without needing HTML/CSS.

Semester Execution Plan

1. **Data Sourcing and Preparation:** Weeks 1-3. Students pull a dataset from Kaggle. Using Pandas, they clean the data, handle missing values, and perform exploratory data analysis (EDA). The deliverable is a clean dataset ready for training.
2. **Predictive Machine Learning:** Weeks 4-7. Using Scikit-learn, students train classification models to discover potential customers. For example, predicting a "High Intent to Purchase" flag based on past browsing behavior. They evaluate their models using standard metrics like precision, recall, and F1-score.
3. **Developing the vAI Analyst:** Weeks 8-11. This is where the LLM integration happens. Students write Python scripts to pass the outputs of their Scikit-learn models (along with the customer's profile data) into a local LLM via Ollama. They must engineer prompts that instruct the LLM to act as a BI analyst.
4. **Dashboard Deployment:** Weeks 12-14. Students wrap the entire pipeline in a Streamlit interface. The final application should allow a user to select a customer ID, view the ML prediction metrics, and read the LLM's generated strategy.

Demonstrating the Advantage of vAI

To ensure the project clearly demonstrates the value of a Virtual AI, grade the final deliverables on how well the system bridges the gap between raw data and human action.

Traditional BI requires a human analyst to interpret a chart and decide what to do. In this project, the vAI handles the cognitive load:

- **Traditional BI Output:** "Customer #405: 82% probability of conversion."
- **vAI BI Output:** "Customer #405 has an 82% conversion probability. They have frequently browsed the electronics category but abandoned their cart twice. **Action item:** The system recommends sending a 10% discount code specifically for wireless headphones to close the sale."

This stark contrast perfectly illustrates to students how integrating LLMs with traditional machine learning transforms data from a passive report into an active business asset.

1 Introduction

This reference guide provides a structured blueprint for instructors implementing the Virtual AI (vAI) Customer Intelligence Engine project. It organizes the semester using a multi-dimensional engineering framework, ensuring students understand not just the code, but the complete architecture from data ingestion to business narrative.

1.1 The Pedagogical Framework

To ensure students grasp the full scope of modern AI engineering, the course milestones can be structured across five interconnected layers:

1. **Fundamental Layer (Weeks 1-3):** Students ground their work in probability, statistics, and data structures. They select a dataset, perform Exploratory Data Analysis (EDA), and establish the baseline metrics for customer behavior.
2. **Technology Layer (Weeks 4-7):** The core machine learning implementation. Students utilize **Pandas**, **NumPy**, and **Scikit-learn** to build predictive classification models (e.g., Random Forest or XGBoost) to identify high-potential customers or churn risks.
3. **System Layer (Weeks 8-10):** The integration phase. Students bridge their predictive models with generative AI, passing statistical outputs and profile data as contextual prompts into a local LLM framework.
4. **Application Layer (Weeks 11-12):** Deploying the solution as a tangible product. Students encapsulate their Python pipelines into an interactive web application using **Streamlit**, creating a functional dashboard for end-users.
5. **Narrative Layer (Weeks 13-14):** The business translation. The final evaluation focuses on the vAI's ability to interpret the raw data and generate a clear, strategic narrative (e.g., recommending specific marketing interventions) that a non-technical manager can act upon.

1.2 Technical Environment Setup

To keep infrastructure complexity low and ensure data privacy, the recommended stack relies entirely on local execution and open-source frameworks:

- **Data & ML:** Python environment configured with Scikit-learn, Pandas, and NumPy.
- **The vAI Engine: Ollama** is highly recommended for this course. It allows students to run models like Llama 3 or Mistral locally on their own machines, completely eliminating the need for cloud API keys or internet-dependent inference.
- **Interface: Streamlit** allows students to build the Application Layer using pure Python, removing the need to teach HTML/CSS/JavaScript within a single semester.

1.2.1 Recommended Datasets

Students should source data that requires both mathematical prediction and behavioral explanation. Excellent Kaggle options include:

- **Customer Churn & Segmentation Dataset (Synthetic):** Contains 2,000 records of e-commerce data with features like spending scores and membership levels.
- **Customer Behavior & Churn Prediction:** A 100,000-record dataset simulating e-commerce behavior with non-linear relationships and missing values for robust EDA practice.
- **Direct-to-Consumer E-Commerce Funnel:** Tracks ~120,000 user sessions across a marketing funnel, perfect for predicting conversion drop-offs.

1.3 Course Timeline & Evaluation Matrix

Adjust the interactive timeline below to see how shifting the project's emphasis (Predictive ML vs. Generative LLM) impacts the weekly milestones and the final grading rubric.

1.4 Final Academic Deliverables

Beyond the functional Streamlit application, students should produce professional documentation that mirrors academic and industry standards.

Require students to document their methodology, model performance metrics, and system architecture in a formal project report. Utilizing typesetting systems like **Typst** or publishing frameworks like **Quarto** ensures their final manuscripts are presentation-ready and effectively communicate the engineering process from the fundamental math to the final narrative output. See Knuth (1984)

2 Summary

Here is the reference Python code mapped to the four phases of the semester timeline. These scripts assume the students have installed the necessary libraries (`pandas`, `scikit-learn`, `ollama`, `streamlit`) and have a local LLM (like `llama3`) running via Ollama.

2.1 Weeks 1-3: Data Sourcing and Preparation

In this phase, students write a script to load their Kaggle dataset, handle missing values, encode categorical variables, and save the clean data for machine learning.

```
# data_prep.py
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

# 1. Load Kaggle dataset (e.g., Customer Churn Data)
df = pd.read_csv('customer_data.csv')

# 2. Exploratory Data Analysis & Cleaning
# Fill missing numerical values with the median
df.fillna(df.median(numeric_only=True), inplace=True)

# 3. Encode categorical features into numerical values for ML
encoder = LabelEncoder()
df['Membership_Type'] = encoder.fit_transform(df['Membership_Type'])
df['Contract_Length'] = encoder.fit_transform(df['Contract_Length'])

# 4. Separate features (X) and the target variable (y)
X = df.drop('Churn', axis=1) # Target variable
y = df['Churn']

# 5. Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Save the cleaned datasets for the next phase
X_train.to_csv('X_train.csv', index=False)
X_test.to_csv('X_test.csv', index=False)
y_train.to_csv('y_train.csv', index=False)
y_test.to_csv('y_test.csv', index=False)

print("Data preparation complete. Clean datasets saved.")
```

2.2 Weeks 4-7: Predictive Machine Learning

Students build and evaluate a Scikit-learn classification model to discover "at-risk" or "high-potential" customers. The model is saved as a `.pkl` file so it can be used later in the final application.

```
# ml_model.py
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score
import joblib

# 1. Load the prepared training data
X_train = pd.read_csv('X_train.csv')
y_train = pd.read_csv('y_train.csv').values.ravel()
X_test = pd.read_csv('X_test.csv')
y_test = pd.read_csv('y_test.csv').values.ravel()

# 2. Initialize and train the Random Forest Classifier
print("Training the ML model...")
clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X_train, y_train)

# 3. Evaluate the model predictions
y_pred = clf.predict(X_test)
print(f"Model Accuracy: {accuracy_score(y_test, y_pred) * 100:.2f}%\n")
print("Classification Report:\n", classification_report(y_test, y_pred))

# 4. Save the trained model for the Application Layer
joblib.dump(clf, 'customer_intelligence_model.pkl')
print("Model saved as 'customer_intelligence_model.pkl'")
```

2.3 Weeks 8-11: Developing the vAI Analyst

Students learn prompt engineering and API integration. They write a script that passes the customer's raw data and the ML prediction into a locally hosted LLM via the official `ollama` Python library.

```python
# vai_analyst.py
import ollama

def generate_customer_strategy(customer_profile: dict, prediction_score: float) -> str:
    """
    Passes the ML prediction and customer data to a local LLM via Ollama.
    """
    # 1. Construct the prompt with strict instructions for the LLM
    prompt = f"""
    You are an expert Business Intelligence vAI Analyst.
    Review the following customer profile and their machine learning churn probability.

    Customer Profile Data:
    {customer_profile}

    Algorithm Churn Risk Prediction: {prediction_score * 100:.1f}%

    Task: Write a concise, 3-sentence strategy for the marketing team detailing
    why this customer might churn based on their profile and what specific action
    (e.g., discount, check-in call) to take to retain them. Do not use filler words.
    """

    # 2. Call the local LLM model (e.g., Llama 3 or Mistral)
    try:
        response = ollama.generate(model='llama3', prompt=prompt)
        return response['response']
    except Exception as e:
        return f"Error connecting to local vAI: {e}"

# Example Usage to test the pipeline
if __name__ == "__main__":
```

```
    sample_data = {'Age': 34, 'Membership_Type': 'Basic', 'Total_Spend': 120.50, 'Support_Ti
    risk_score = 0.82

    strategy = generate_customer_strategy(sample_data, risk_score)
    print("--- vAI Strategy Output ---\n")
    print(strategy)
```

2.4 Weeks 12-14: Dashboard Deployment

Students bring everything together using Streamlit. This code loads the saved Scikit-learn model, scores a customer on the fly, and uses the `ollama.chat` function with `stream=True` to create a real-time, ChatGPT-style typing effect on the dashboard.

```
# app.py
# Run this file in the terminal using: streamlit run app.py

import streamlit as st
import pandas as pd
import joblib
import ollama

st.set_page_config(page_title="vAI Customer Intelligence", layout="wide")
st.title("vAI Customer Intelligence Engine")

# 1. Load the saved model and test dataset into the app
@st.cache_resource
def load_assets():
    model = joblib.load('customer_intelligence_model.pkl')
    data = pd.read_csv('X_test.csv')
    return model, data

model, df = load_assets()

# 2. Build the sidebar for user selection
st.sidebar.header("Intelligence Dashboard")
customer_idx = st.sidebar.selectbox("Select Customer ID to Analyze", df.index)
selected_customer = df.loc[customer_idx]
```

```
# 3. Display the raw data
st.subheader("Raw Customer Data")
st.dataframe(pd.DataFrame([selected_customer]))

# 4. Generate the ML Prediction
st.subheader("Algorithm Prediction")
# predict_proba returns a 2D array, we extract the probability of class 1 (churn/convert)
ml_prob = model.predict_proba(selected_customer.values.reshape(1, -1))[0][1]

# Display the metric with dynamic coloring
if ml_prob > 0.6:
    st.error(f"High Risk Prediction: {ml_prob * 100:.1f}%")
else:
    st.success(f"Low Risk Prediction: {ml_prob * 100:.1f}%")

# 5. Connect to the vAI Analyst
st.markdown("---")
st.subheader("vAI Strategic Analyst")

if st.button("Generate Action Plan"):
    with st.spinner("vAI is analyzing the profile..."):

        prompt = f"Profile: {selected_customer.to_dict()} | Risk Score: {ml_prob:.2f}. Give a

        # Create an empty container to hold the streaming text
        response_container = st.empty()
        full_response = ""

        # Stream the LLM response directly to the Streamlit UI
        for chunk in ollama.chat(model='llama3', messages=[{'role': 'user', 'content': prompt
            full_response += chunk['message']['content']
            response_container.markdown(full_response + " ")

        # Final render without the cursor block
        response_container.markdown(full_response)
```

References

Knuth, Donald E. 1984. "Literate Programming." *Comput. J.* (USA) 27 (2): 97–111. https://doi.org/10.1093/comjnl/27.2.97.